

Cláusula / Operación	Definición y Propósito	Sintaxis Básica	Ejemplo de Uso
ORDER BY	Ordena los resultados de la consulta según una o varias columnas, ya sea de forma ascendente (ASC) o descendente (DESC).	SELECT col FROM tabla ORDER BY col [ASC DESC];	SELECT * FROM All_numbers ORDER BY val DESC;
LIMIT	Restringe la cantidad máxima de filas que devuelve la consulta. Opcionalmente acepta un desplazamiento (OFFSET).	SELECT col FROM tabla LIMIT n [OFFSET m];	SELECT * FROM All_numbers LIMIT 5; <i>(devuelve solo los primeros 5 registros)</i>
GROUP BY	Agrupar filas que tienen los mismos valores en columnas especificadas, utilizado frecuentemente con funciones de agregación (COUNT, SUM, AVG).	SELECT col, COUNT(*) FROM tabla GROUP BY col;	SELECT tipo, COUNT(*) FROM Numeros GROUP BY tipo;
INNER JOIN	Devuelve únicamente las filas donde existe una coincidencia exacta de la condición entre ambas tablas (Intersección $\cap$).	SELECT * FROM T1 INNER JOIN T2 ON T1.id = T2.id;	SELECT * FROM Even INNER JOIN Odd ON Even.val = Odd.val;
LEFT JOIN	Devuelve todas las filas de la tabla izquierda y las coincidencias de la derecha; si no hay coincidencia, llena con NULL.	SELECT * FROM T1 LEFT JOIN T2 ON T1.id = T2.id;	SELECT * FROM All_nums LEFT JOIN Odd ON All_nums.val = Odd.val;
CROSS JOIN	Produce el producto cartesiano entre dos tablas, combinando cada fila de la primera tabla con todas las filas de la segunda.	SELECT * FROM T1 CROSS JOIN T2;	SELECT * FROM Even CROSS JOIN Odd; <i>(genera 25 combinaciones para 5 pares y 5 impares)</i>

Notas de implementación en SQLite

- **Secuencia de Ejecución:** El motor de SQLite procesa las consultas en este orden conceptual: FROM / JOIN $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY $\rightarrow$ LIMIT.
- **Combinación de JOINS:** LEFT JOIN combinado con WHERE tabla_derecha.id IS NULL se utiliza en SQLite para simular la diferencia de conjuntos (Anti-Join).